

In [1]:

```
#import relevant modules
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import math
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.metrics import mean_absolute_error
```

In [2]:

```
#Locate the csv file
data = 'D:\\AI Course\\Data Engineering\\country_vaccinations.csv'
```

In [3]:

```
#read the file into the workbook
Data = pd.read_csv(data)
```

In [4]:

```
#Create a dataframe
Vacc_Data = pd.DataFrame(Data)
Vacc_Data
```

Out[4]:

	country	iso_code	date	total_vaccinations	people_vaccinated	people_fully_vaccinated	daily_vaccinations_raw	daily_vaccinations	total_v
0	Afghanistan	AFG	22/02/2021	0.0	0.0	NaN	NaN	NaN	
1	Afghanistan	AFG	23/02/2021	NaN	NaN	NaN	NaN	1367.0	
2	Afghanistan	AFG	24/02/2021	NaN	NaN	NaN	NaN	1367.0	
3	Afghanistan	AFG	25/02/2021	NaN	NaN	NaN	NaN	1367.0	
4	Afghanistan	AFG	26/02/2021	NaN	NaN	NaN	NaN	1367.0	
...	...	...	...	...	...	...	...	...	
86507	Zimbabwe	ZWE	25/03/2022	8691642.0	4814582.0	3473523.0	139213.0	69579.0	
86508	Zimbabwe	ZWE	26/03/2022	8791728.0	4886242.0	3487962.0	100086.0	83429.0	
86509	Zimbabwe	ZWE	27/03/2022	8845039.0	4918147.0	3493763.0	53311.0	90629.0	
86510	Zimbabwe	ZWE	28/03/2022	8934360.0	4975433.0	3501493.0	89321.0	100614.0	
86511	Zimbabwe	ZWE	29/03/2022	9039729.0	5053114.0	3510256.0	105369.0	103751.0	

86512 rows × 10 columns

In [5]:

```
#see if there is null data
.isnull().sum()
```

Out[5]:

```
country          0
iso_code         0
date             0
total_vaccinations    42905
people_vaccinated    45218
people_fully_vaccinated 47710
daily_vaccinations_raw 51150
daily_vaccinations      299
total_vaccinations_per_hundred    42905
people_vaccinated_per_hundred    45218
people_fully_vaccinated_per_hundred 47710
daily_vaccinations_per_million    299
vaccines          0
source_name       0
source_website    0
dtype: int64
```

In [6]:

```
#remove all null data
Vacc_Data.dropna(inplace=True)
```

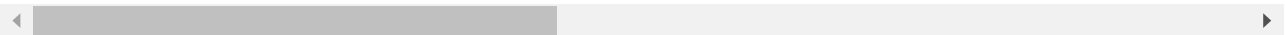
In [7]:

```
Vacc_Data
```

Out[7]:

	country	iso_code	date	total_vaccinations	people_vaccinated	people_fully_vaccinated	daily_vaccinations_raw	daily_vaccinations	total_vaccinations_per_hundred
94	Afghanistan	AFG	27/05/2021	593313.0	479574.0	113739.0	2859.0	6487.0	100.0
101	Afghanistan	AFG	03/06/2021	630305.0	481800.0	148505.0	4015.0	5285.0	100.0
339	Afghanistan	AFG	27/01/2022	5081064.0	4517380.0	3868832.0	6868.0	9802.0	100.0
433	Albania	ALB	18/02/2021	3049.0	2438.0	611.0	1348.0	254.0	100.0
515	Albania	ALB	11/05/2021	622507.0	440921.0	181586.0	9548.0	12160.0	100.0
...	...	...	...	...	...	...	...	...	...
86507	Zimbabwe	ZWE	25/03/2022	8691642.0	4814582.0	3473523.0	139213.0	69579.0	100.0
86508	Zimbabwe	ZWE	26/03/2022	8791728.0	4886242.0	3487962.0	100086.0	83429.0	100.0
86509	Zimbabwe	ZWE	27/03/2022	8845039.0	4918147.0	3493763.0	53311.0	90629.0	100.0
86510	Zimbabwe	ZWE	28/03/2022	8934360.0	4975433.0	3501493.0	89321.0	100614.0	100.0
86511	Zimbabwe	ZWE	29/03/2022	9039729.0	5053114.0	3510256.0	105369.0	103751.0	100.0

30847 rows × 10 columns



In [8]:

```
Vacc_Data.columns
```

Out[8]:

```
Index(['country', 'iso_code', 'date', 'total_vaccinations',
      'people_vaccinated', 'people_fully_vaccinated',
      'daily_vaccinations_raw', 'daily_vaccinations',
      'total_vaccinations_per_hundred', 'people_vaccinated_per_hundred',
      'people_fully_vaccinated_per_hundred', 'daily_vaccinations_per_million',
      'vaccines', 'source_name', 'source_website'],
      dtype='object')
```

In [9]:

```
#remove columns that are redundant
Vacc_Data.drop(['iso_code', 'vaccines', 'source_name', 'source_website'], axis=1, inplace=True)
```

In [10]:

Vacc_Data

Out[10]:

	country	date	total_vaccinations	people_vaccinated	people_fully_vaccinated	daily_vaccinations_raw	daily_vaccinations	total_vaccinations
94	Afghanistan	27/05/2021	593313.0	479574.0	113739.0	2859.0	6487.0	
101	Afghanistan	03/06/2021	630305.0	481800.0	148505.0	4015.0	5285.0	
339	Afghanistan	27/01/2022	5081064.0	4517380.0	3868832.0	6868.0	9802.0	
433	Albania	18/02/2021	3049.0	2438.0	611.0	1348.0	254.0	
515	Albania	11/05/2021	622507.0	440921.0	181586.0	9548.0	12160.0	
...	...	...	...	...	...	...	...	
86507	Zimbabwe	25/03/2022	8691642.0	4814582.0	3473523.0	139213.0	69579.0	
86508	Zimbabwe	26/03/2022	8791728.0	4886242.0	3487962.0	100086.0	83429.0	
86509	Zimbabwe	27/03/2022	8845039.0	4918147.0	3493763.0	53311.0	90629.0	
86510	Zimbabwe	28/03/2022	8934360.0	4975433.0	3501493.0	89321.0	100614.0	
86511	Zimbabwe	29/03/2022	9039729.0	5053114.0	3510256.0	105369.0	103751.0	

30847 rows × 11 columns

In [11]:

```
#begin to create my model and identify what I will use to as the feature variables
feature_data = Vacc_Data[["total_vaccinations", "people_vaccinated", "people_fully_vaccinated",
                           "daily_vaccinations_raw", "daily_vaccinations", "total_vaccinations_per_hundred",
                           "people_vaccinated_per_hundred", "daily_vaccinations_per_million"]]
```

In [12]:

```
#identify the target variable and what we want to predict
target_data = Vacc_Data[["people_fully_vaccinated_per_hundred"]]
```

In [13]:

```
#assign x and y to make the test simpler to understand
x = feature_data
y = target_data
```

In [14]:

```
#begin to create the model and train it up
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=4)
print(x_test.shape)
print(x_train.shape)
```

```
(6170, 8)
(24677, 8)
```

In [15]:

```
#now to create the linear regression model
linreg = LinearRegression()
linreg.fit(x_train, np.ravel(y_train))
```

Out[15]:

LinearRegression()

In [16]:

```
#assign what we want our linear regression to target
y_pred = linreg.predict(x_test)
```

In [17]:

```
#calculate MSE to see the magnitude of errors
from sklearn.metrics import mean_squared_error

MSE = mean_squared_error(y_test,y_pred)
print(MSE)
```

24.189544051423308

In [18]:

```
#calculate MAE to see the frequency of errors
from sklearn.metrics import mean_absolute_error

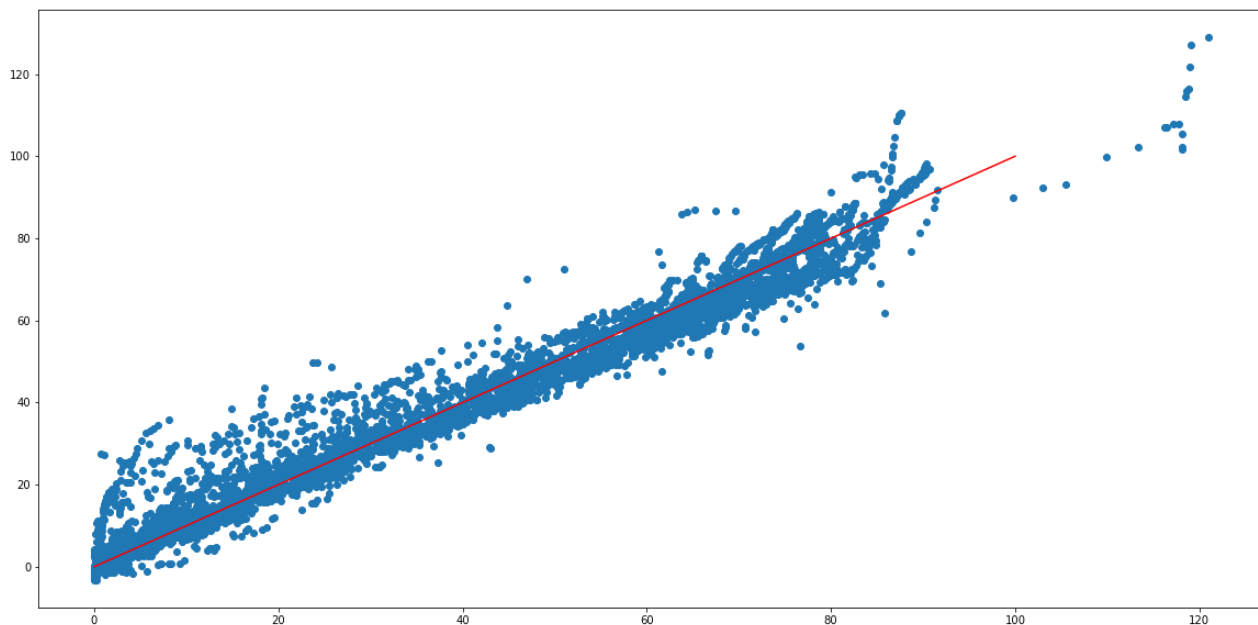
MAE = mean_absolute_error(y_test,y_pred)
print(MAE)
```

3.3988779244144744

In [29]:

```
#create a scatter plot to show the predicted result against the actual results

plt.figure(figsize=(20,10))
plt.scatter(x=y_test, y=y_pred)
plt.plot([0,100],[0,100], color='r')
plt.show()
```



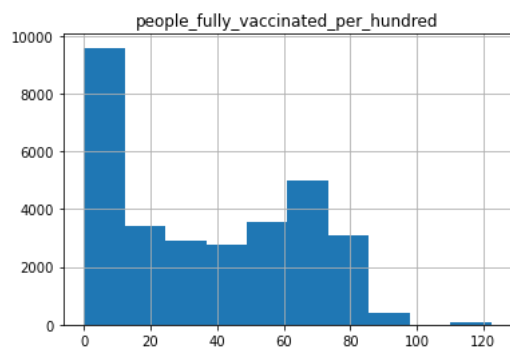
In [20]:

```
#using a histogram we can see the frequency of people vaccinated per hundred

Vacc_Data.hist(column='people_fully_vaccinated_per_hundred')
```

Out[20]:

```
array([[<AxesSubplot:title={'center':'people_fully_vaccinated_per_hundred'}>]],
      dtype=object)
```

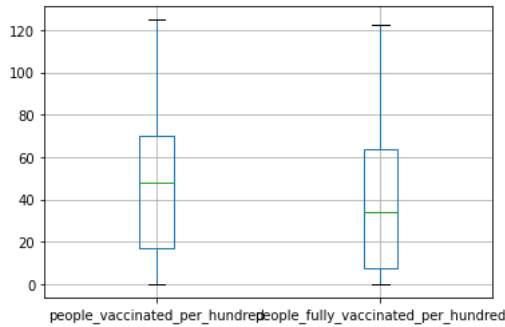


In [21]:

```
V_plot = Vacc_Data.boxplot(column= ['people_vaccinated_per_hundred', 'people_fully_vaccinated_per_hundred'])
V_plot.plot()
```

Out[21]:

[]



In [22]:

```
#upload the dataframe to SQL
from sqlalchemy import create_engine
engine = create_engine('postgresql:username:password@localhost:/mydatabase')
df.to_sql('Vacc_data', engine)
```

```
-----
ArgumentError                                Traceback (most recent call last)
<ipython-input-22-de072d58889c> in <module>
      1 from sqlalchemy import create_engine
----> 2 engine = create_engine('postgresql:username:password@localhost:Kieran1998!/mydatabase')
      3 df.to_sql('Vacc_data', engine)

~\anaconda3\lib\site-packages\sqlalchemy\engine\_init_.py in create_engine(*args, **kwargs)
    498     strategy = kwargs.pop("strategy", default_strategy)
    499     strategy = strategies[strategy]
--> 500     return strategy.create(*args, **kwargs)
    501
    502

~\anaconda3\lib\site-packages\sqlalchemy\engine\strategies.py in create(self, name_or_url, **kwargs)
     52     def create(self, name_or_url, **kwargs):
     53         # create url.URL object
--> 54         u = url.make_url(name_or_url)
     55
     56         plugins = u._instantiate_plugins(kwargs)

~\anaconda3\lib\site-packages\sqlalchemy\engine\url.py in make_url(name_or_url)
    227
    228     if isinstance(name_or_url, util.string_types):
--> 229         return _parse_rfc1738_args(name_or_url)
    230     else:
    231         return name_or_url

~\anaconda3\lib\site-packages\sqlalchemy\engine\url.py in _parse_rfc1738_args(name)
    288     return URL(name, **components)
    289     else:
--> 290     raise exc.ArgumentError(
    291         "Could not parse rfc1738 URL from string '%s'" % name
    292     )

ArgumentError: Could not parse rfc1738 URL from string 'postgresql:username:password@localhost:Kieran1998!/mydatabase'
```

In []: